LIVING WORLD

# 経済・アイテム循環設計書

資産保存則、採集・製造・市場・銀行・物流・死亡・ダンジョン

## 文書管理

| 項目 | 内容 |
| --- | --- |
| 文書ID | ECO-SPEC-001 |
| 版 | Ver.1.0 / 実装レビュー用 |
| 対象 | 採択済み LIVING WORLD プロジェクト |
| 基準日 | 2026年10月21日 |
| 参照方針 | ［合意済］＝これまでの構想／［設計案］＝実装推奨／［要決定］＝未承認 |

## 本書の読み方

本書は実装担当が仕様レビューとタスク分解に使用するための専門設計書である。数値、採用技術、実装優先順位のうち未承認の事項は「設計案」「要決定」と明記した。全項目は検証・実測により改訂する。

本書に記載された設計案は、ゲーム実装済み・性能達成済みを意味しない。関連するAPI・DB・バランス設計書と突き合わせて確定する。

# 目次・要求追跡

章IDは関連資料・課題管理・テストケースから参照できる。各章は「合意済み事項」「実装案」「処理フロー」「主要データ」「異常系」「受入基準」「要決定」を含む。


01 ECO-01 資産保存則と台帳の原則

02 ECO-02 資源採集と再生

03 ECO-03 製造・レシピ・職人技能

04 ECO-04 地域市場・価格形成

05 ECO-05 通貨・銀行・信用

06 ECO-06 物流・重量・保険

07 ECO-07 死亡・ダンジョン・戦利品プール

08 ECO-08 公共事業・都市財政・物価連鎖

09 ECO-09 長期均衡・市場悪用対策


## 横断的な品質原則

・サーバーが世界状態と資産台帳の正本を保持し、クライアントやLLMの発言だけでは変更できない。

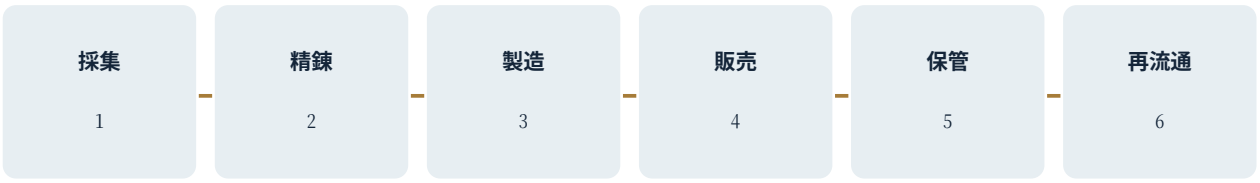
・イベントには一意IDと因果関係を付け、二重処理・通信切断・再試行に耐える。

・プレイヤーの行動結果は社会・経済・NPCへ波及するが、無限連鎖・無限資産生成は防ぐ。

・仕様の確定値・仮値・要決定を混在させず、レビュー履歴を残す。

# ECO-01 資産保存則と台帳の原則

人工物の無からの生成を禁止し、資産移動は二重記帳・単一コミットで追跡する。




## 合意済みのゲーム仕様

・武器・防具・薬などの人工物は採集・加工・製造によってのみ生じる。

・死亡した携行品は地面・ダンジョン・モンスター所持等へ移り、複製されない。

## 実装・アルゴリズム案

・アイテムはitem_instance_idとitem_definition_idを分離。数量スタックは一意のロットIDを持つ。

・所有者・場所・状態は1つだけ。通貨は口座残高だけでなく仕訳台帳を正本とする。

## 処理手順（イベント順序）

1. 採集イベントで自然資源を合法的に生成
2. 製造は原材料の消費と成果物生成を同一取引に記録
3. 売買は所有権と通貨を原子的に交換
4. 死亡は所有者を遺品プールへ移管
5. 監査で台帳の投入・流出・現在量を突合

## 主要データ・インターフェース

・item_instance(id, definition_id, owner_type, owner_id, location_id, version)

・item_transfer(event_id, from, to, quantity, reason)

・ledger_entry(tx_id, account_id, amount, currency)

## 異常系・競合時の挙動

・二重送信→idempotency_keyで1回のみ確定。

・取引途中障害→DBコミット前なら全取消。

## 受入基準・テスト観点

・同一個体が複数所有者に存在しない。

・すべての人工物に製造または世界初期配備の起源イベントがある。

## 要決定・レビュー論点

・世界初期配備在庫は初期資産として明示し、以後の人工物自動補充は禁止。

| レビュー項目 | 確認内容 |
| --- | --- |
| 仕様追跡 | ECO-01 をタスク、テスト、変更履歴に紐付ける |
| 担当の確認 | 実装担当・ゲームデザイナー・QAが条件と異常系を承認する |

# ECO-02 資源採集と再生

天然資源の再生は許容するが、供給速度と地理条件で制約する。

## 合意済みのゲーム仕様

・天然資源は再生し得る。人工物は自然復活しない。

## 実装・アルゴリズム案

・採掘ノードには最大埋蔵量・回復速度・季節・採掘難度・所有権を設定。

・採集結果は資源残高の減算と素材ロット生成を同一トランザクションにする。

## 処理手順（イベント順序）

1. 採集権・道具・重量を確認
2. ノード残量をロック
3. 採集量を計算し残量を減算
4. 素材ロットをインベントリへ追加
5. 重量超過なら地面保管等の明示的な処理へ

## 主要データ・インターフェース

・resource_node(id, biome, stock, capacity, regen_rule, owner_id)

・harvest_event(node_id, qty, tool_durability_cost)

## 異常系・競合時の挙動

・同時採掘→行ロック／楽観ロックで残量超過を禁止。

・採集後に接続切断→確定した採集物は台帳に残す。

## 受入基準・テスト観点

・資源残高が0未満にならない。

・再生で武器・完成薬が直接生まれない。

## 要決定・レビュー論点

・採集速度・季節変動・資源の再生上限は数値調整対象。

| レビュー項目 | 確認内容 |
| --- | --- |
| 仕様追跡 | ECO-02 をタスク、テスト、変更履歴に紐付ける |
| 担当の確認 | 実装担当・ゲームデザイナー・QAが条件と異常系を承認する |

# ECO-03 製造・レシピ・職人技能

加工工程ごとに材料・設備・技能・時間を検証する。

## 合意済みのゲーム仕様

・鍛冶師などの非戦闘職は修行で成長する。

・製造物は現実の材料から生産される。

## 実装・アルゴリズム案

・レシピはinput_items/output_items/byproducts/tool_wear/skill_requirement/workstation/timeで表現。

・製造失敗時の材料損失と副産物も保存則に従って記録する。

## 処理手順（イベント順序）

1. レシピ・職人技能・設備を確認
2. 材料ロットを製造予約へ移す
3. 製造時間と品質判定を実行
4. 材料を消費し完成品と副産物を生成
5. 熟練度・工具耐久を更新

## 主要データ・インターフェース

・recipe_id / version / input_bill / output_bill / craft_duration

・craft_job(status, reserved_items, quality_seed, result_event)

## 異常系・競合時の挙動

・職人ログアウト→作業の継続・中断ポリシーを適用。

・設備破壊→予約材料を返還または損失として記帳。

## 受入基準・テスト観点

・同じ材料が2件の製造に予約されない。

・品質と成功判定を固定シードで再現できる。

## 要決定・レビュー論点

・製造時間・失敗率・品質の幅は検証用初期値とする。

| レビュー項目 | 確認内容 |
| --- | --- |
| 仕様追跡 | ECO-03 をタスク、テスト、変更履歴に紐付ける |
| 担当の確認 | 実装担当・ゲームデザイナー・QAが条件と異常系を承認する |

# ECO-04 地域市場・価格形成

地域別に在庫・需要・輸送費・危険・情報遅延を持たせる。

## 合意済みのゲーム仕様

・冒険者都市で剣需要が増えると価格が上がる等、地域ごとの需給で価格が変わる。

## 実装・アルゴリズム案

・価格案 P_next=clamp(P_prev×(1+α×(demand−supply)/max(supply, ε)),P_min,P_max)。

・価格はイベント駆動で更新し、無限の自動買取・自動販売は設けない。

## 処理手順（イベント順序）

1. 購入注文・販売注文・在庫変化を収集
2. 一定条件で市場清算を起動
3. 在庫・注文・基準価格から参考価格を更新
4. 実際の約定は在庫と残高を確認
5. 地域価格と取引履歴を公開範囲に応じ配信

## 主要データ・インターフェース

・market(region_id,item_def_id,stock,reference_price,version)

・order(order_id,side,qty,limit_price,expires_at)

・trade(buyer,seller,qty,price,tax,delivery)

## 異常系・競合時の挙動

・価格ゼロ・供給ゼロ→上下限と在庫不足で取引不可。

・大量注文による操作→注文制限と市場監視を実施。

## 受入基準・テスト観点

・在庫不足の約定が起きない。

・地域間で価格差が発生し、輸送費を含めて裁定可能。

## 要決定・レビュー論点

・α・価格上下限・NPCの購入力は長期シミュレーションで調整。

| レビュー項目 | 確認内容 |
| --- | --- |
| 仕様追跡 | ECO-04 をタスク、テスト、変更履歴に紐付ける |
| 担当の確認 | 実装担当・ゲームデザイナー・QAが条件と異常系を承認する |

# ECO-05 通貨・銀行・信用

銀行の保管は安全、輸送中の物資には危険が残るという違いを明示する。

## 合意済みのゲーム仕様

・中規模以上の都市・村に安全な銀行貸金庫がある。

・銀行に預けた物は死亡・襲撃によって失われない。

## 実装・アルゴリズム案

・銀行保管はinventory_location=BANK_VAULTとし、個人携行品から原子的に移す。

・銀行間送金と現物配送を分離し、現物は到着までin_transitとして追跡する。

## 処理手順（イベント順序）

1. 預入権限と容量・手数料を確認
2. 携行品を銀行ロケーションへ移管
3. 手数料を台帳に計上
4. 引出し時に重量と空き枠を検証
5. 別都市配送は配送契約と危険判定を開始

## 主要データ・インターフェース

・bank_vault(owner_id,branch_id,capacity,fee_policy)

・bank_deposit(item_id, vault_id, tx_id)

・money_account / ledger_entry

## 異常系・競合時の挙動

・死亡処理と預入競合→同一資産行を排他制御。

・銀行支店停止→安全な保管を維持し操作だけ延期。

## 受入基準・テスト観点

・銀行内の物が死亡ドロップに含まれない。

・通貨移動の借方・貸方が一致する。

## 要決定・レビュー論点

・銀行の手数料、融資、預金利息、信用制度は未確定。

| レビュー項目 | 確認内容 |
| --- | --- |
| 仕様追跡 | ECO-05 をタスク、テスト、変更履歴に紐付ける |
| 担当の確認 | 実装担当・ゲームデザイナー・QAが条件と異常系を承認する |

# ECO-06 物流・重量・保険

徒歩・馬・荷馬・馬車・船で移動速度、積載量、費用、危険が異なる。

## 合意済みのゲーム仕様

・重量・容積制限があり、超過すると移動や戦闘に不利になる。

・運送は襲撃や事故のリスクを持ち得る。

## 実装・アルゴリズム案

・配送はcargo_manifestにロット・所有権・保険・護衛契約を持たせる。

・重量・容積・経路・治安・天候からコストと所要時間を算出する設計案。

## 処理手順（イベント順序）

1. 出荷前に荷物を予約・梱包
2. 運送手段と経路を選択
3. 費用と保険を決済
4. 輸送中イベント（襲撃・遅延）を評価
5. 到着で受領者に所有権を移管

## 主要データ・インターフェース

・shipment(id,origin,destination,carrier,manifest,status)

・cargo_lot(item_id,weight,volume,insured_value)

・shipment_event(type,location,time)

## 異常系・競合時の挙動

・襲撃→紛失した現物は犯人・地面等へ移り複製しない。

・配達先不在→支店保管・再配達・返送を選択。

## 受入基準・テスト観点

・貨物が発送元と到着先の双方に同時存在しない。

・積載量超過を拒否する。

## 要決定・レビュー論点

・保険金原資・支払上限・事故頻度は制度設計が必要。

| レビュー項目 | 確認内容 |
| --- | --- |
| 仕様追跡 | ECO-06 をタスク、テスト、変更履歴に紐付ける |
| 担当の確認 | 実装担当・ゲームデザイナー・QAが条件と異常系を承認する |

# ECO-07 死亡・ダンジョン・戦利品プール

携行品の死亡移転とダンジョン戦利品の流入を一つの台帳で管理する。

## 合意済みのゲーム仕様

・死亡時に携行品を失い、レベルが2低下する。現金の一部も失う。

・遺品は地面・ダンジョン宝箱・モンスター等へ再配置され得る。

## 実装・アルゴリズム案

・death_eventに携行品スナップショットを固定し、分配先を決めて移転する。

・宝箱は実在する在庫プールから補充し、在庫のない高級武器を新規生成しない。

## 処理手順（イベント順序）

1. 死亡処理で携行資産を凍結
2. 死亡ペナルティと銀行保護を適用
3. 資産を遺品プールへ移管
4. 分配規則で地面・敵・宝箱に割当
5. 拾得時に遺品プールから受取人へ移管

## 主要データ・インターフェース

・death_event(id,player_id,level_before,level_after)

・loot_pool(pool_id,item_id,source_event,assigned_location)

・chest_stock(chest_id,item_id,qty)

## 異常系・競合時の挙動

・二重死亡イベント→同一death_idは一度だけ処理。

・敵が装備を拾う→実在装備を敵インベントリへ移し戦力に反映。

## 受入基準・テスト観点

・同一死亡品が複数宝箱へ複製されない。

・長期未探索ダンジョンは在庫が蓄積し得る。

## 要決定・レビュー論点

・現金損失率・再配置率・消失条件・回収期限は要決定。

| レビュー項目 | 確認内容 |
| --- | --- |
| 仕様追跡 | ECO-07 をタスク、テスト、変更履歴に紐付ける |
| 担当の確認 | 実装担当・ゲームデザイナー・QAが条件と異常系を承認する |

# ECO-08 公共事業・都市財政・物価連鎖

城壁破壊が資材・賃金・治安・財政に波及する仕組みを定義する。

## 合意済みのゲーム仕様

・城壁の損壊から賠償・修理依頼・材料需要・物価・襲撃危険に連鎖する。

## 実装・アルゴリズム案

・都市予算をtax_revenue / treasury / debt / public_contractで管理。

・公共事業は資金確保後にギルドへ発注し、現物資材と労働が投入されて初めて修復が進む。

## 処理手順（イベント順序）

1. 城壁損壊で防御値・賠償額を更新
2. 市政が修理見積りと資金調達を実施
3. ギルド契約を作成し資材を発注
4. 地域市場で石材・輸送・労働需要が上昇
5. 実際の投入量に応じ城壁耐久を回復

## 主要データ・インターフェース

・city_treasury / damage_claim / public_work_order

・repair_material_consumption / city_defense_index

## 異常系・競合時の挙動

・市の資金不足→段階修理・増税・借入等の政策候補。

・プレイヤーが逃亡→未収債権として保持。

## 受入基準・テスト観点

・材料未投入で城壁耐久が回復しない。

・支払済みの契約報酬が二重払いされない。

## 要決定・レビュー論点

・自治体の借入・破産・徴税のルールは別途承認。

| レビュー項目 | 確認内容 |
| --- | --- |
| 仕様追跡 | ECO-08 をタスク、テスト、変更履歴に紐付ける |
| 担当の確認 | 実装担当・ゲームデザイナー・QAが条件と異常系を承認する |

# ECO-09 長期均衡・市場悪用対策

在庫と通貨の流入出を監視し、価格操作や経済崩壊を早期に検出する。

## 合意済みのゲーム仕様

・NPCも消費・生産・交易に参加し、都市の成長・衰退に影響する。

## 実装・アルゴリズム案

・経済監視指標：通貨総量、貨幣回転率、必需品充足率、価格変動率、取引集中度、失業率。

・チート・多重アカウントによる価格操作と通常の交易をログで区別する。

## 処理手順（イベント順序）

1. ゲーム内30/90/365日を自動進行
2. 市場と生活必需の在庫・収支を集計
3. インフレ・飢餓・独占の警報を評価
4. 税・輸送・生産調整の政策案をシミュレート
5. バランス変更はバージョン管理し再現テスト

## 主要データ・インターフェース

・economic_snapshot(day,region,price_index,money_supply)

・trade_concentration / npc_unmet_needs / bankruptcies

## 異常系・競合時の挙動

・価格暴騰→取引を隠して調整せず、世界イベントとして供給・輸送改善を促す。

## 受入基準・テスト観点

・人工物総量の変化が製造・破壊・初期配備で説明できる。

・貨幣総量の変化が正規の発行・回収イベントで説明できる。

## 要決定・レビュー論点

・通貨の新規発行元・税の使途・経済危機の救済規則を確定する。

| レビュー項目 | 確認内容 |
| --- | --- |
| 仕様追跡 | ECO-09 をタスク、テスト、変更履歴に紐付ける |
| 担当の確認 | 実装担当・ゲームデザイナー・QAが条件と異常系を承認する |

# 付録A：資産台帳の不変条件

- INV-01：各item_instance_idの有効な所有者・場所は常に一つ。
- INV-02：人工物の生成にはcraft_completedまたはinitial_world_seedの起源イベントが必須。
- INV-03：通貨の振替は同一tx_idの借方・貸方が釣り合う。
- INV-04：銀行保管物は死亡移転の対象外。
- INV-05：輸送中の現物は発送元・到着先の同時所有を禁止。
- INV-06：宝箱からの取得はダンジョン在庫プールを必ず減算。
- INV-07：再送・タイムアウト後の再試行はidempotency_keyで同じ結果を返す。

# レビュー・変更管理チェックリスト

- □ 合意済み・設計案・要決定が正しく区別されているか。
- □ 機能間で資産・NPC・クエスト・地域の参照整合性が成立するか。
- □ 正常系・異常系・再送・同時実行の受入テストがあるか。
- □ 負荷・推論費・運営監視の予算を測定できるか。
- □ プレイヤーへの説明とゲーム内の実挙動に矛盾がないか。
- □ 仕様変更時にAPI・DB・UI・バランス・テストを同時更新するか。

## 未確定事項の管理

各章の「要決定」は課題管理システムで所有者・期限・承認者・決定理由を付けて管理する。承認後は本書の版を更新し、互換性を壊す変更はマイグレーション計画と回帰テストを必須とする。